

AI (Lec- 7)

Note Title

21-01-2021

STATE SPACE SEARCH

State :- Current configuration of agent.
Abstract representation of agent's environment.

Initial State :- Description of starting configuration of agent.

Action/Operator :- takes agent from one state to another.

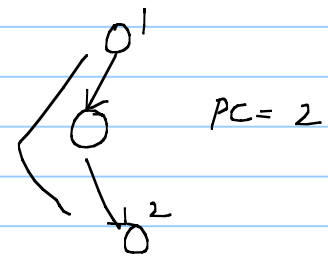
A state can have many successor states.

Plan :- Sequence of Actions.

Goal :- Description of set of desirable states of world.

Path Cost :- Path $\Rightarrow$ +ve number

Usually sum of steps as path cost



Problem Formulation:- Choosing a relevant set of states to consider and a feasible set of operators for moving from one state to another.

Search:- Process of imagining sequence of operators applied to initial state and checking which sequence reaches goal state.

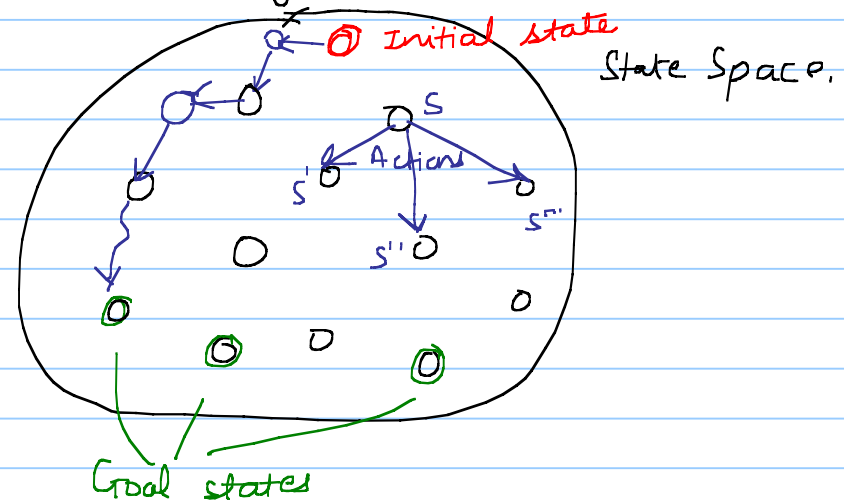
Search problem

S : full set of states

S_0 : initial state

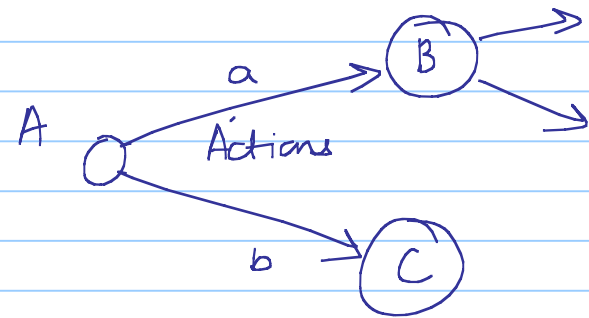
A : $S \rightarrow S$ set of operators

G : Set of final states $G \subseteq S$



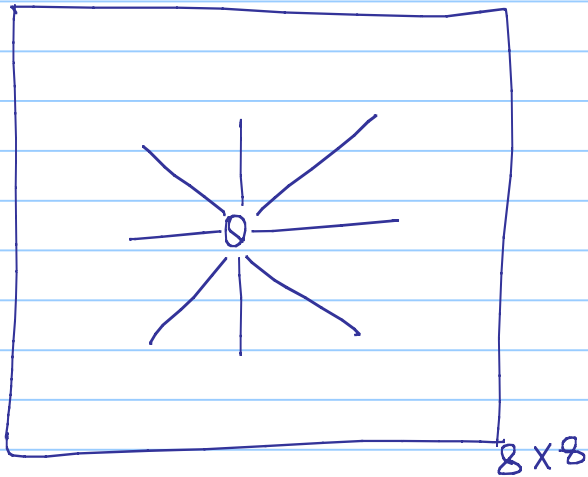
Searching Process:

- Checks the current state.
- Executes allowable actions to move to next state.
- Check if new state is goal state
if not then the new state becomes current state & the process is repeated until we reach goal state.



8-Queens Problem

Place 8 queens on a chessboard so that no two queens are in same row, column or diagonal.



Problem Formulation 1

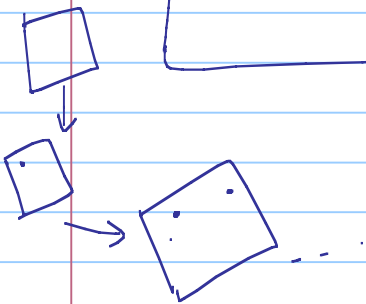
States :- Any arrangement of 0 to 8 queens on the board.

Initial state :- 0 queens on the board.

Goal state :- 8 queens on the board, none are attacked.

Action / Operators :- Adding a queen in board /
Changing position of existing queen.

0 → 8



Problem Formulation 2

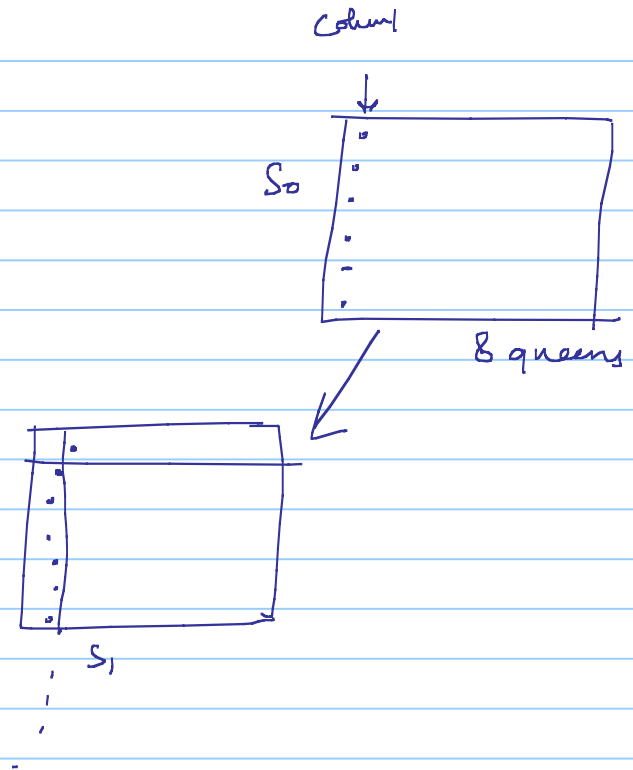
States :- Any arrangement of 8 queens on board.

Initial state :- All queens are in column 1.

Goal state :- 8 queens on board with no attack.

Action :- Change the position of any queen.

8 queens $\rightsquigarrow$ 8 queens



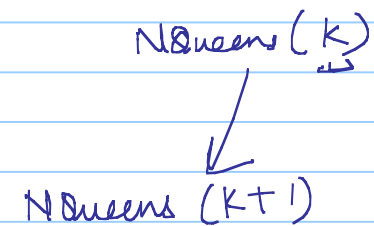
Problem Formulation 3

States:- Any arrangement of K queens in first K rows such that none are attacked.

Initial state :- 0 queens on board. $K=0$

Goal state :- 8 queens on board, $K=8$

Action:- Add a queen in $(K+1)$ th row such that none are attacked.



8 Puzzle Problem

↓

3	2	
5	1	6
4	8	7

Goal state :-

✓ ↓

1	2	3
4	5	6
7	8	

↓ ✓

1	4	7
2	5	8
3	6	

↓ ✓

1	2	3
8		4
7	6	5

State :- Description of each 8 tiles in each location it occupies.

→ S1(1,2,3,4,5,6,7,8,B)

← S2(1,4,7,2,5,8,3,6,B)

← S3(1,2,3,8,B,4,7,6,5)

Goal states

Action :-

←

5	4
6	1 8
7	3 2

5	4
6	1 8
7	3 2

←

5	4	8
6		1
7	3	2

↖ ↗

5	4	8
6		1
7	3	2

↓ ✗

5	4	8
6		1
7	3	2

Water Jug Problem

✓

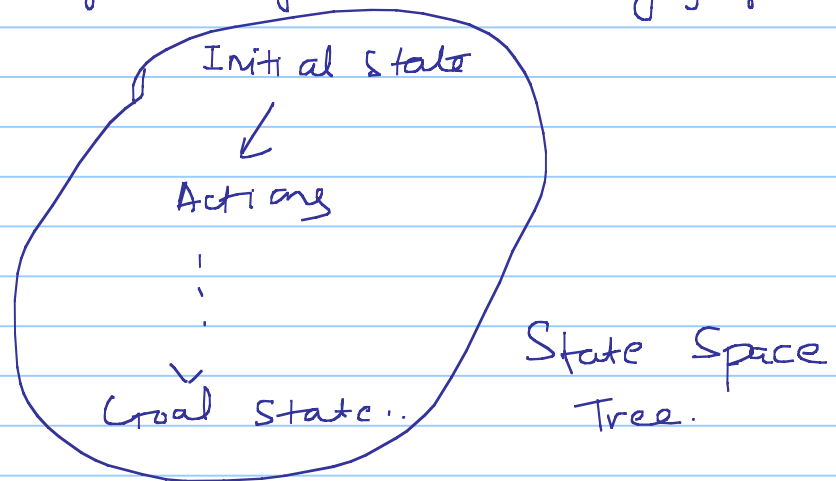
You have three jugs: measuring 12 gallons, 8 gallons, 3 gallons. & water faucet. You need to measure exactly 1 gallon of water in any jug.

State Representation ?

Initial state ? ✓

Goal states ?

Action / operators ?



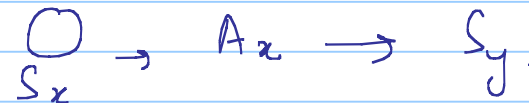
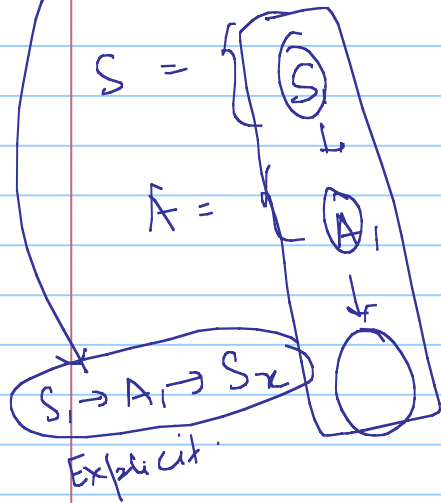
Explicit v/s Implicit Search Space



Explicitly all the states/ operators are represented

Implicit Representation.
&

generated when required.



AI Lecture

Note Title

25-01-2021

Water Jug Problem

You have three jugs measuring 12 gallons, 8 gallons and 3 gallons and a water faucet. You need to measure exactly one gallon of water in any jug.

Initially: All jugs are empty

Goal: Some jug contains exactly one gallon.

State Space Representation

① State :- $\overset{J_1}{x}, \overset{J_2}{y}, \overset{J_3}{z}$, $0 \leq x \leq 12, 0 \leq y \leq 8, 0 \leq z \leq 3$

② Initial state :- $(0, 0, 0)$

③ Goal state(s) :- $(1, y, z), (x, 1, z), (x, y, 1)$

Initial (0, 0, 0)

12, 8, 3

Actions/ Operators :-

$$x - \min(8, y, 8-y)$$

$$x < 8, y = 0 \rightarrow 8$$

$$y < 8$$

$$y + \min(8, x, 8-x) = y$$

$$\begin{cases} 10. (x, y, z) \rightarrow (0, \min(8, x+y), z) \\ 11. (x, y, z) \rightarrow (0, y, \min(3, x+z)) \end{cases}$$

1. $(0, y, z) \rightarrow (12, y, z)$ { Filling of 1st jug }
2. $(x, y, z) \rightarrow (12, y, z)$ { Filling of 1st jug }
3. $(x, 0, z) \rightarrow (x, 8, z)$ { Filling of 2nd jug }
4. $(x, y, 0) \rightarrow (x, y, 3)$ { " " 3rd " }
5. $(x, y, z) \rightarrow (0, y, z)$ { Empty 1st jug }
6. $(x, y, z) \rightarrow (x, 0, z)$ { " 2nd " }
7. $(x, y, z) \rightarrow (x, y, 0)$ { " 3rd " }
8. $(x, y, z) \rightarrow (x - \min(x, 8-y), \min(8, y+x), z)$ { Pour from J1 to J2 }
9. $(x, y, z) \rightarrow (x - \min(x, 3-z), y, \min(3, x+z))$ { J1 → J3 }
10. $(x, y, z) \rightarrow (0, \min(8, x+y), z)$ { J1 → J2 }
11. $(x, y, z) \rightarrow (0, y, \min(3, x+z))$ { J1 → J3 }

empty

Search through a State Space

I/p :- Set of states
operators / actions [and costs]
Initial / Start state
Goal state .

O/p :- Path (start  goal state)
= state
[may be shortest path or
path with smallest cost]

Search Algorithm

1- Let L be a list containing the initial state ($L = \text{fringe}$)

2. Loop

if L is empty return fail

Node $\leftarrow$ **select** (L)

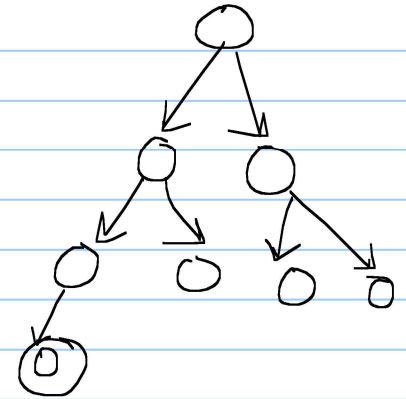
if Node is Goal state then return Node. (or path.)

else

expand Node (apply all applicable actions on Node)

Merge the newly generated states into fringe.

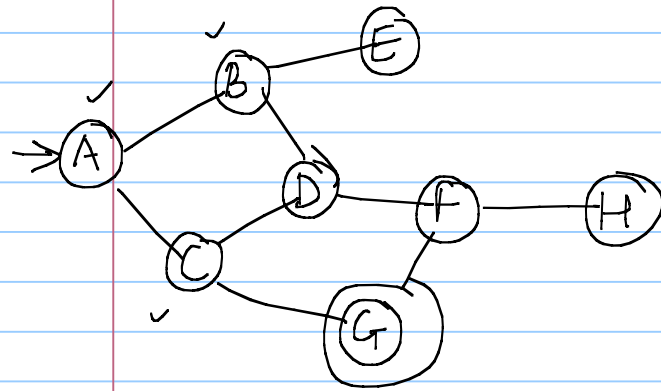
3- End.



Search Strategy

- Completeness :- Is it guaranteed to find a solⁿ if exists?
- Optimality :- Does the solⁿ has optimal / minimal cost?
- Complexity (Time/Space) :-

Search Tree



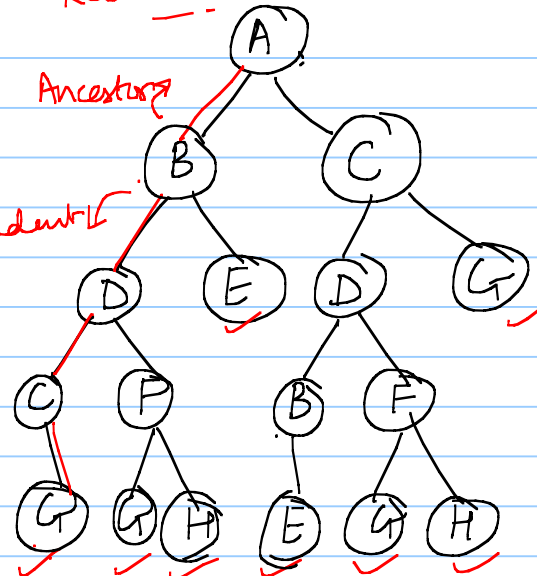
Branching factor is 2
(max possible child node)

Complete path / Partial path :-

Root node

Ancestors

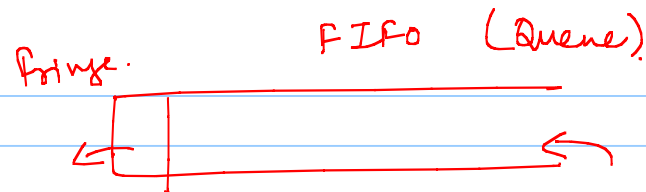
Descendants



Search Tree

✓ : Leaf nodes

Breadth First Search



1- Let L be a list

2- Loop

if L is empty return fail

Node $\leftarrow$ Remove-first (fringe)

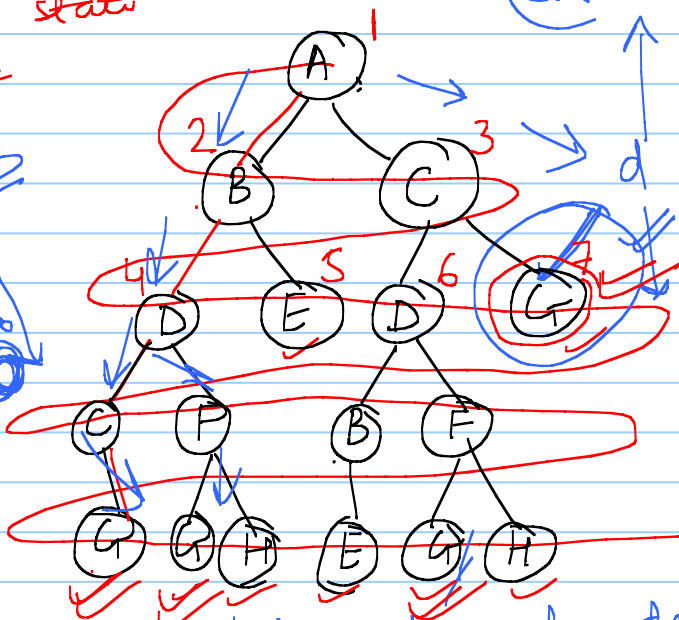
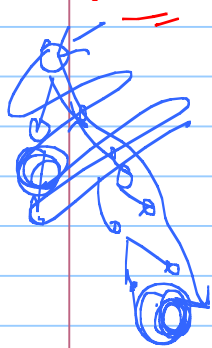
if Node is Goal state then return Node. (or path.)

else

generate all successors of node and add them at back of fringe

3- End.

G is goal state



(expand shallow node first)

BFS

Fringe: ~~A~~ ~~B~~ ~~C~~ ~~D~~ ~~E~~ ~~D~~ ~~G~~ C F B F

1. Complete or not? ✓

Complete

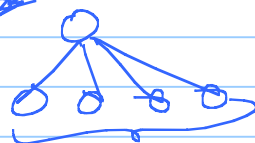
2. Optimal or not? ✓

Optimal

3. Time & Space complexity

$O(b, d, m)$?

d : depth of shallowest goal node.
 m : max depth of search tree.

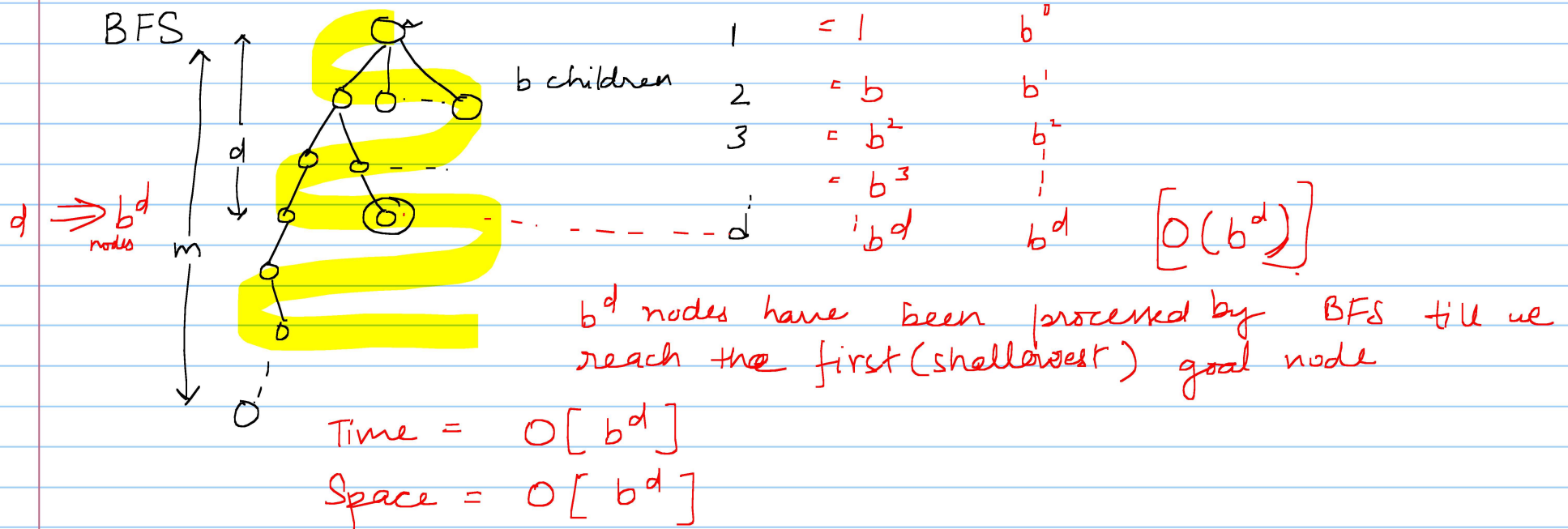


b : branching factor

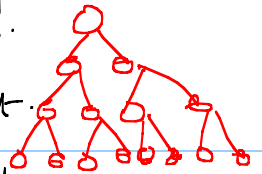
AI (Lec-9)

Note Title

27-01-2021

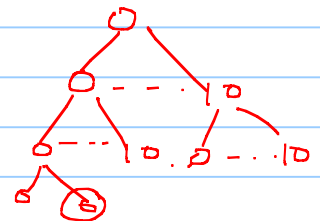


Advantage :- Find path of minimal length to goal.
 Disadvantage :- Exponential time & space requirement.
 BFS → Complete
 → Optimal
 → Exponential time and space complexity $O(b^d)$ where
 d : depth of solution.
 b : branching factor



Let consider a complete search tree of depth 12. where every node at depth 0, ..., 11 have 10 children and every node at depth 12 has 0 children.

How many nodes are there? = $O(10^{12})$



If BFS expands 1000 nodes/sec. and each node uses 100 bytes of storage.

→ BFS will take 35 years to run in worst case
 " " " use 111 TeraBytes of memory.

Depth First Search (DFS)

let fringe be a list containing initial state

Loop

if fringe is empty return failure

Node $\leftarrow$ Remove-first(fringe)

if Node is a goal

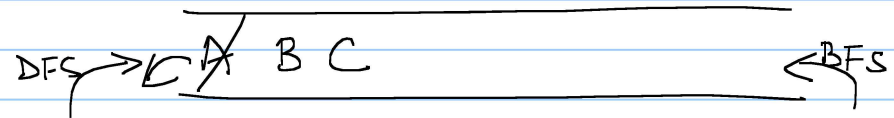
return path from initial state to Node

else

generate all successors of Node and

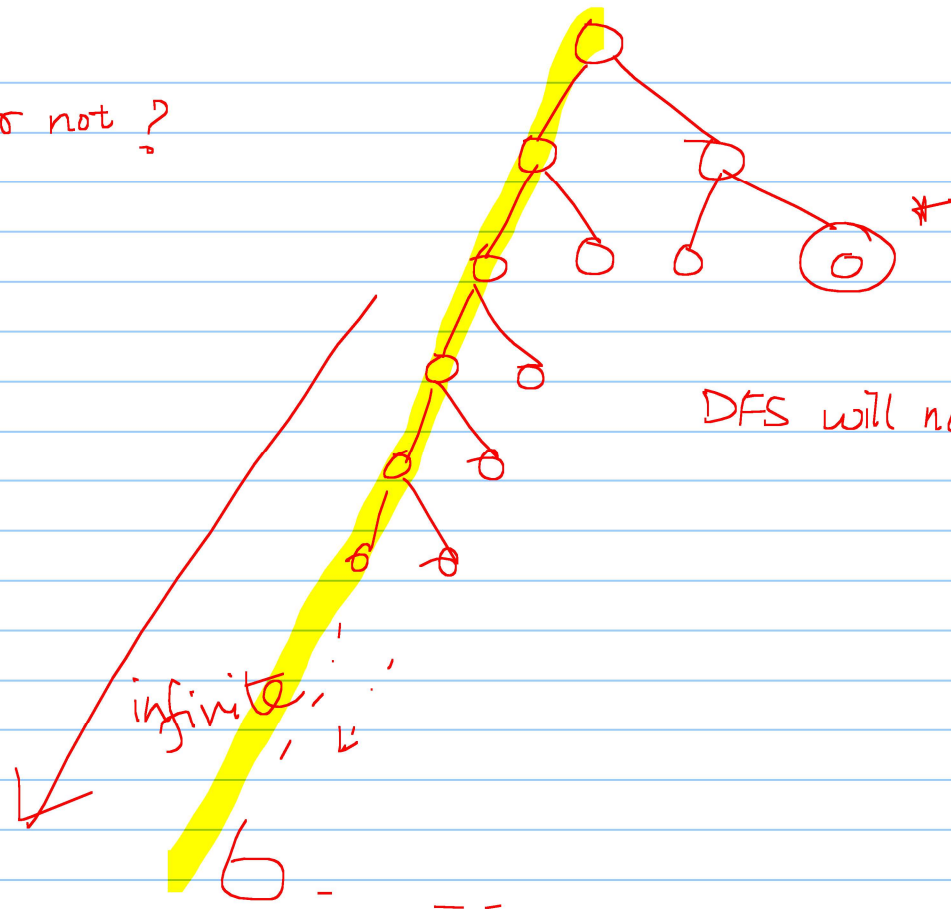
add generated nodes to the front of fringe

End Loop



DFS is complete or not?

↳ not complete



Level 2

DFS will not return a solution.
=.

Depth Limit Search

DFS

(expanding shallowest node first)

limit = 10

Depth Limit Search

Let fringe be a list containing initial state

Loop

if fringe is empty return failure

Node $\leftarrow$ remove-first (fringe)

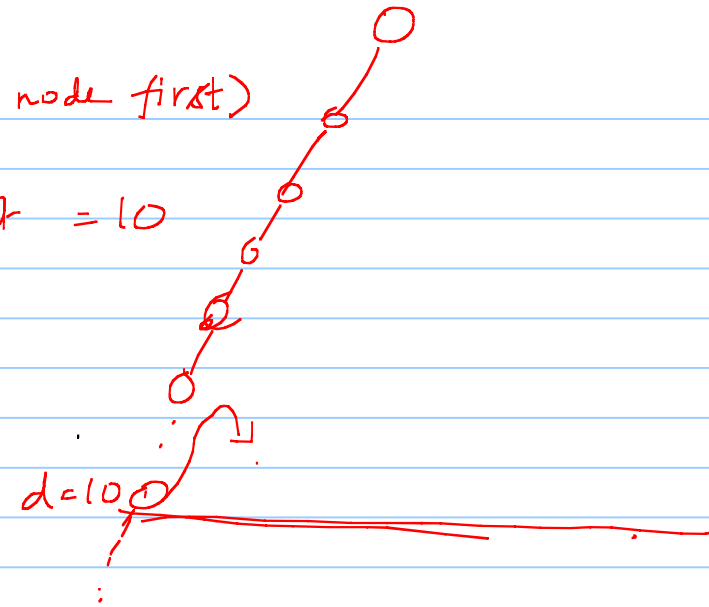
if Node is a goal

return path from initial state to Node

$\Rightarrow$ else if depth of Node = limit return cutoff

else add generated nodes to front of fringe.

End Loop



Complete
= =

(DFID)

Depth-First Iterative Deepening Search

First run DFS on depth 0 (treat that root node has no successors).

If no solution exists then do run DFS on depth 1 and so on.

until solution found do

[DFS with depth cutoff c
 $c = c + 1$

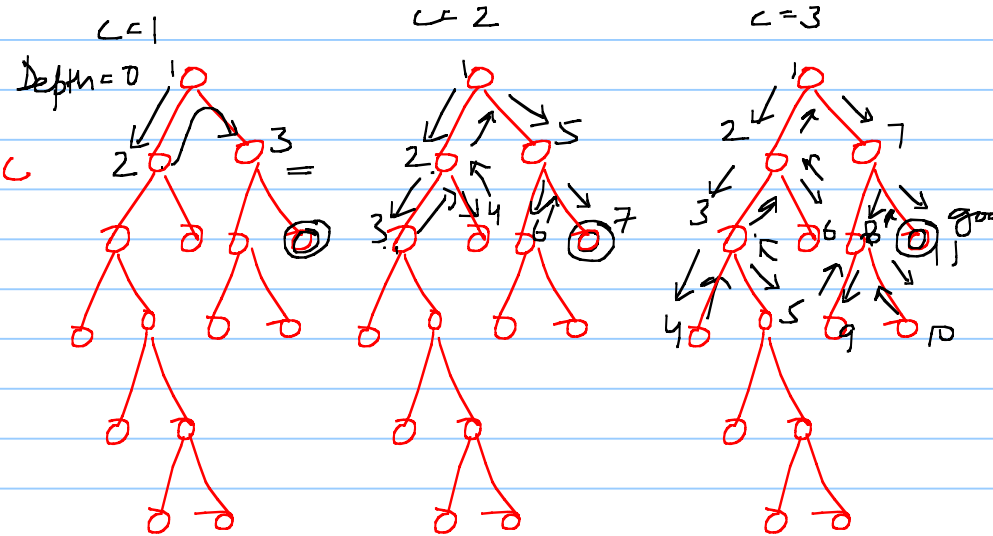
end

DFID search :- Complete ✓

Optimality ? ✓

Time : worse than BFS/DFS

Space (linear) $= O(bd)$ =.



AI

Note Title

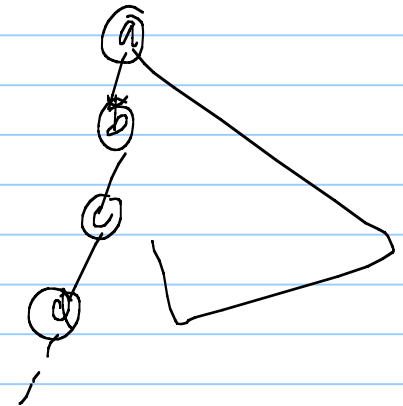
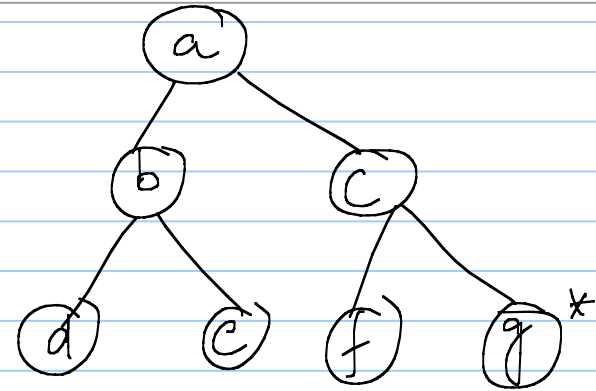
01-02-2021

DFS

↳ Order of processing nodes

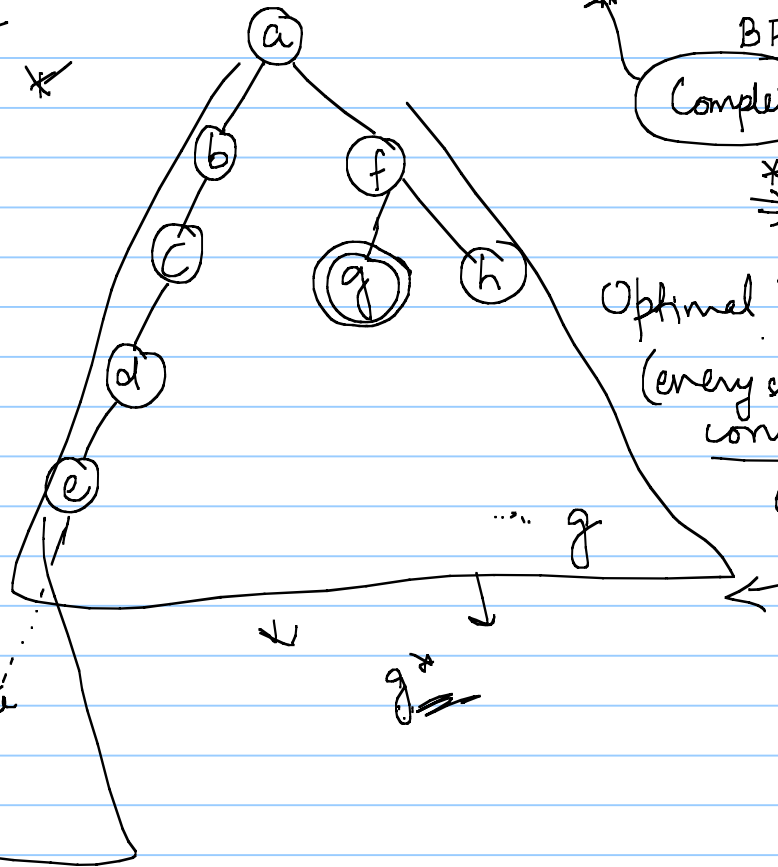
↳ $a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow f \rightarrow g^*$

Complete? No (if depth is infinite)



reaches goal node if it exists (reachable) in finite time.

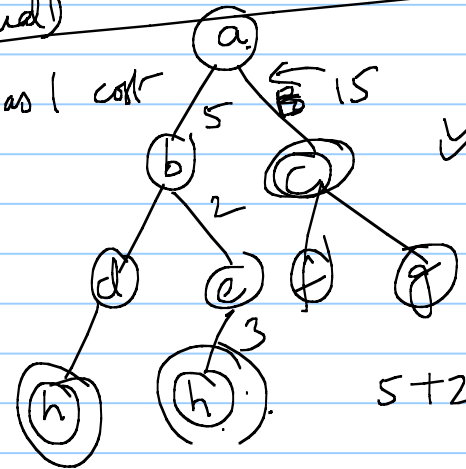
BFS *
complexity *
optimal =



BFS { DFS
Complete? Yes No
* (when b is finite)

Optimal? Yes. No
(every step cost is constant/equal)
every step as 1 cost

Optimality
the path to goal node should be optimal



1) a - c ? c=1
2) a - b - e - h
c=3
5+2+3=10

Time/Space complexity

BFS

DFS

b : branching factor

$O(b^d)$

Time complexity : $O(b^m)$

d : depth of shallowest goal nodes.

Space complexity : $O(\underline{b^m})$:

level 0 $\Rightarrow 1$

level 1 $\Rightarrow b$

level 2 $\Rightarrow b^2$

level 3 $\Rightarrow b^3$

